

Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



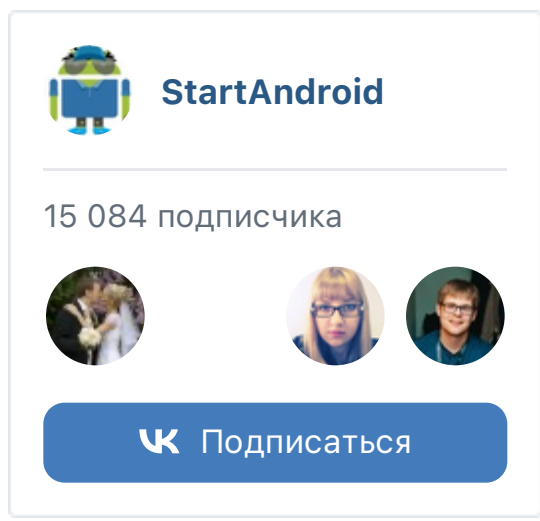
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 7. Сабкомпоненты

21 июня 2021

В это уроке разбираемся, что такое сабкомпоненты

Ранее в примерах у нас всегда был один компонент, который мы создавали в Application классе. В целом, иногда можно обойтись и одним компонентом на все приложение. Но использование нескольких компонентов дает определенные преимущества.

Во-первых - это логическое разделение функционала. Вместо одного компонента, который умеет и знает все, мы создаем разные компоненты, ответственные за свои области.

Во-вторых - это более гибкая передача объектов в компоненты.

В-третьих - это более гибкое время жизни объектов, возвращаемых компонентами.

В этом уроке мы поговорим про первое преимущество. А по двум другим будут отдельные уроки.

Сабкомпоненты

Сабкомпоненты позволяют разделить функционал компонента на несколько частей. Непросто объяснить это словами, поэтому давайте сразу рассмотрим пример.

Пусть у нас есть такой компонент:

```
1 @Component(modules = {StorageModule::class, NetworkModule::class})
2 interface AppComponent {
3     fun getDatabaseHelper(): DatabaseHelper
4     fun getNetworkUtils(): NetworkUtils
5 }
```

Он умеет возвращать нам два объекта. О том, как эти объекты создавать, он знает из модулей.

Мы добавляем в проект MainActivity и хотим, чтобы компонент умел создавать для нас MainActivityPresenter.

Создаем модуль с Provides методом, в котором создается презентер:

```
1 @Module
2 class MainModule {
3
4     @Provides
5     fun provideMainActivityPresenter(databaseHelper: DatabaseHelper, networkUtils: NetworkUtils)
6         return MainActivityPresenter(databaseHelper, networkUtils)
7     }
8
9 }
```

В компоненте добавляем этот модуль в список модулей и описываем метод, который будет возвращать презентер:

```
1 @Component(modules = {StorageModule::class, NetworkModule::class, MainModule::class})
2 interface AppComponent {
3     fun getDatabaseHelper(): DatabaseHelper
4     fun getNetworkUtils(): NetworkUtils
5
6     fun getMainActivityPresenter(): MainActivityPresenter
7 }
```

Теперь в MainActivity мы можем вызвать этот метод компонента и получить презентер:

```
1 appComponent.getMainActivityPresenter()
```

Все вроде в порядке, но что если нам нужен не только презентер, но и еще какие-то объекты, специфичные для MainActivity и его дочерних фрагментов. В этом случае в AppComponent нам надо будет описывать несколько getMain или injectMain методов. А потом будут добавляться еще Activity и для них тоже надо будет создавать методы. В итоге AppComponent превратится в монстра, который умеет и знает все. Такие объекты обычно называют god object. Это признак плохого кода.

Чтобы AppComponent не стал таким, мы можем создать для него сабкомпонент MainComponent. Туда мы вынесем все, что касается MainActivity.

Интерфейс для сабкомпонента создается точно так же, как и для компонента, только аннотация используется другая - @Subcomponent:

```
1 @Subcomponent(modules = {MainModule::class})
2 interface MainComponent {
3     fun getMainActivityPresenter(): MainActivityPresenter
4 }
```

В интерфейсе мы описываем get метод для получения презентера. А в списке модулей мы указываем MainModule, чтобы сабкомпонент знал, как этот презентер создается.

Соответственно, из AppComponent мы убираем и MainModule и getMainActivityPresenter. Они там больше не нужны. AppComponent теперь не должен уметь создавать презентер.

Но AppComponent должен уметь создавать MainComponent. Для этого мы добавляем метод getMainComponent:

```
1 @Component(modules = {StorageModule::class, NetworkModule::class})
2 interface AppComponent {
3     fun getDatabaseHelper(): DatabaseHelper
4     fun getNetworkUtils(): NetworkUtils
5
6     fun getMainComponent(): MainComponent
7 }
```

Теперь в MainActivity мы можем получить MainComponent из AppComponent:

```
1 val mainComponent = appComponent.getMainComponent()
```

А из MainComponent можем получить презентер:

```
1 mainComponent.getMainActivityPresenter()
```

Т.е. мы перенесли часть функционала из одного компонента в другой.

Теперь давайте разбираться, как все это работает.

sub

Компоненты AppComponent и MainComponent - это не просто какие-то независимые друг от друга компоненты.

MainComponent - это сабкомпонент для компонента AppComponent.

Для начала надо разобраться с семантикой, чтобы не было недопонимания. Для меня, например, приставка саб- (sub) означает "что-то меньшее". Т.е. это должно означать, что сабкомпонент это какая-то часть от целого компонента. И что сабкомпонент умеет и знает меньше чем его компонент родитель.

На самом деле, все ровно наоборот. Сабкомпонент - это объект, который умеет все то же, что и компонент, его создавший, плюс имеет какой-то свой функционал.

Это похоже на иерархию классов. Когда класс наследник умеет все то же, что и класс родитель. А также наследник имеет и свой функционал, которого в родителе не было.

Связь

В чем заключается связь между компонентом и сабкомпонентом?

Если помните, компонент AppComponent мы создаем вручную, используя билдер (или фабрику) из доступного нам класса DaggerAppComponent, который создается даггером.

Но для сабкомпонентов такой класс не создается. Вернее, у нас нет доступа к этому классу. Поэтому создать сабкомпонент сами мы не можем. Это может сделать только компонент, который является родителем для сабкомпонента.

Когда мы в AppComponent прописали метод, который возвращает MainComponent, мы тем самым обозначили, что компонент AppComponent будет родителем для сабкомпонента MainComponent. И если нам нужен экземпляр MainComponent, мы можем получить его от AppComponent.

Это ведет к тому, что компонент и сабкомпонент тесно связаны друг с другом под капотом. Компонент сам создает класс-реализацию интерфейса сабкомпонента. Т.е. класс компонента (DaggerAppComponent) внутри себя создает класс MainComponentImpl, который является реализацией интерфейса MainComponent.

Эта реализация сабкомпонента (MainComponentImpl) включает в себя ссылку на компонент, который его создал. Т.е. сабкомпонент имеет доступ к компоненту. А если быть совсем точным, то к модулям компонента. Используя модули компонента, сабкомпонент может создать любой объект, доступный компоненту.

Например, мы у сабкомпонента MainComponent можем попросить объект DatabaseHelper:

```
1 @Subcomponent(modules = {MainModule::class})
2 interface MainComponent {
3     fun getMainActivityPresenter(): MainActivityPresenter
4     fun getDatabaseHelper(): DatabaseHelper
5 }
```

Заметьте, что сам MainComponent не знает как его создавать. Мы не прописывали ему модуль StorageModule, где он мог бы это узнать. Но у сабкомпонента MainComponent будет доступ к модулям компонента AppComponent, и у них он сможет попросить этот объект.

Аналогично, если сабкомпоненту для создания своих объектов понадобится какой-то объект, то он обратится к компоненту. Пример с презентером это подтверждает:

```
1 @Module
2 class MainModule {
3
4     @Provides
5     fun provideMainActivityPresenter(databaseHelper: DatabaseHelper, networkUtils: NetworkUtils)
6         return MainActivityPresenter(databaseHelper, networkUtils)
7     }
8
9 }
```

Чтобы MainComponent мог создать презентер, ему нужны DatabaseHelper и NetworkUtils. Сам он их создавать не умеет. Поэтому он просит их у компонента родителя, т.е. у AppComponent.

Повторим еще раз. Сабкомпонент - это не какая-то часть компонента. Это скорее похоже на расширение компонента. У сабкомпонента MainComponent есть свои объекты, которые он умеет создавать (MainActivityPresenter). Но при этом он сможет создать (или использовать для создания своих объектов) любой объект, доступный в AppComponent.

А вот AppComponent ничего не знает про MainActivityPresenter и создать его не сможет.

Еще важный момент. Заметьте, что в сабкомпоненте MainComponent мы нигде и никак не указывали какой именно компонент будет создателем (родителем) этого сабкомпонента. Это может быть AppComponent. А может быть и любой другой компонент. Главное, чтобы он предоставил сабкомпоненту необходимые объекты.

Назад Вперёд

Комментарии

RE: Урок 7. Сабкомпоненты — Alisher 25.04.2022 09:11

Т.е. класс компонента (DaggerAppComponent) внутри себя создает класс MainComponentImpl, который является реализацией интерфейса MainComponent.

" - Правильно ли понимаю, что создаётся внутренний не статический класс?

Ответить

RE: Урок 7. Сабкомпоненты — Dmitry Vinogradov 05.05.2022 11:21

Да, внутренний, чтобы он имел доступ к данным родительского класса (компонента).

Ответить

RE: Урок 7. Сабкомпоненты — Лют 15.03.2023 13:06

Данный урок, яркий пример правильного нейминга.

Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

3Comments

